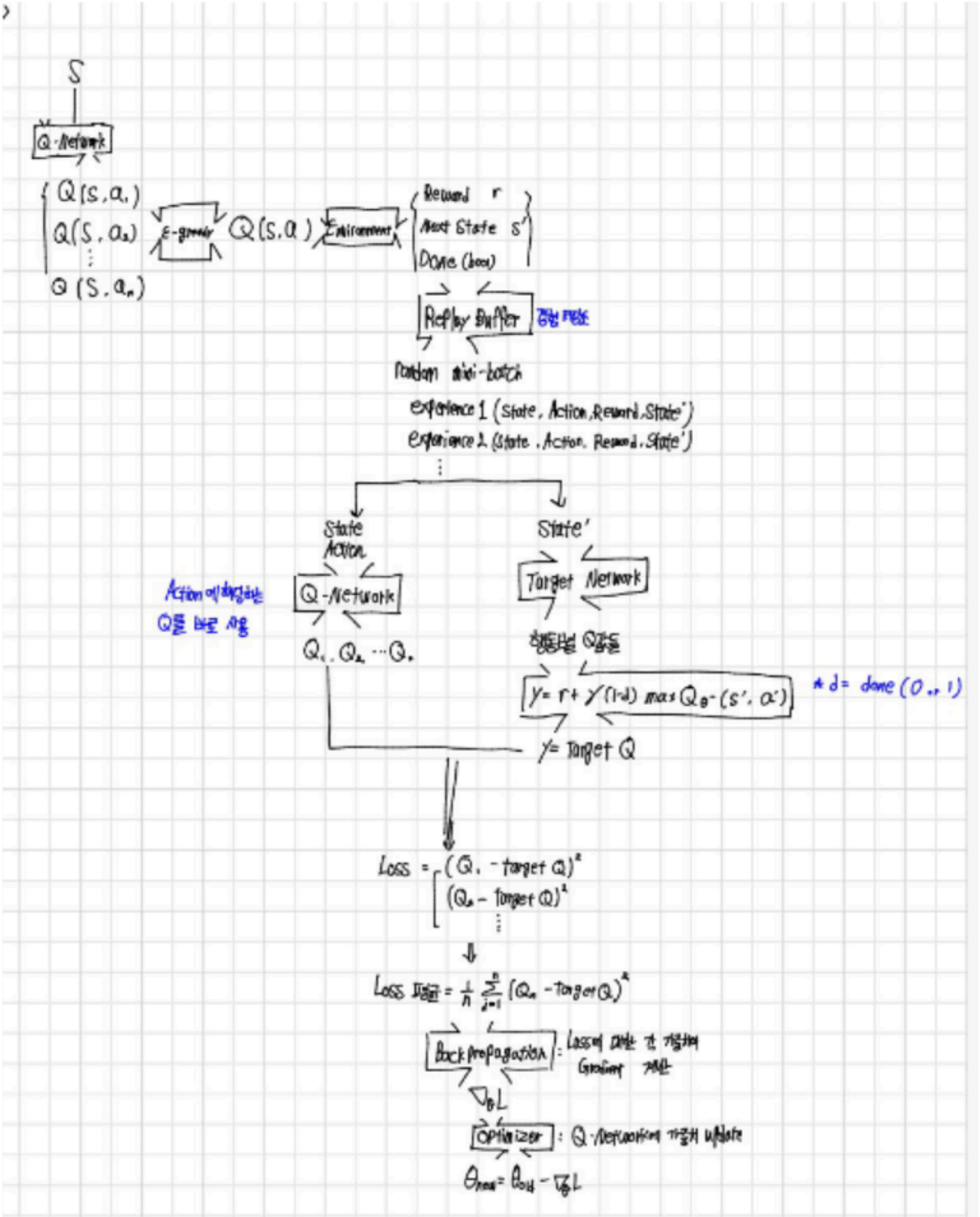
강화학습 Day2 과제1(CartPole)

1. DQN 공부 내용

1.1 DQN workflow 정리



1.2 공부하면서 들었던 의문과 해결

Q1. DQN이 왜 필요한가?

A. Q-learning 만으로는 엄청 오래 걸리는 문제를 Q-Network를 통해 Q값을 빠르게 추정할 수 있게 해주기 때문

Q1-1. Q-learning이랑 DQN이랑 어떻게 다른길래?

A. Q-learning은 Q-table에 저장된 Q값을 경험을 통해 반복적으로 업데이트함.
DQN은 신경망을 통해 Q값을 추정하고 가중치를 업데이트함.

둘다 Q값을 학습하지만 학습하는 방식이 다름

DQN은 예측 Q값과 목표 Q값의 오차를 줄이는 방향으로 가중치를 최적화함.

Q1-2. 결국 둘 다 경험적으로 Q값을 업데이트하는건데 DQN을 사용해야하는 특별한 이유가 있나?

A. Cartpole 문제를 예로 들자면 입력 값 4개가 있다고 할 때
Q-learning은 각 상태 공간을 table에 하나하나 만들어주는 과정이 필요함. 이때 상태 공간을 너무 크게 나누면 상태 정보가 손실되고 너무 작게 나누면 테이블이 너무 커져서 계산이 오래 걸림.
근데 DQN은 학습하지 않은 상태가 입력되더라도 최적화 되어있는 가중치를 이용해 Q값을 한번에 예측할 수 있음.

즉, Q-learning은 Table에 저장된 값을 직접 조회하고 업데이트하는 반면
DQN은 학습된 Neural Network만 있으면 한번에 근사값을 출력할 수 있다는 장점이 있음

Q2. target network랑 Q-network랑 뭐가 다른가?

A. Target Network: 학습 목표인 Q 값을 계산, 행동 가치를 학습함
Q-Network: 현재 Q값을 예측, 학습의 안정화가 목적임

Q2-1. 같은 신경망인데 어떻게 가중치를 최적화하길래 Target Network는 목표 Q값을 출력하고 Q-Network는 예측 Q값을 출력하는가?

A. 동일한 구조이지만 가중치를 업데이트하는 방식이 다르기 때문

우선 학습 과정에서는 Q-Network만 가중치 업데이트를 실행함.
그 다음 출력된 예측Q값과 Target Q값 비교를 반복함.
특정 주기가 되면 지금까지 업데이트된 가중치를 Target Network에 복사함.

즉, Target Network는 Backpropagation, Optimizer 같은 과정 없이 일정 주기로 Q-Network의 가중치를 똑같이 복사해와서 일정 기간 고정된 가중치로 Target Q를 계산해 학습 목표가 계속 변하는 것을 방지하는 역할을 함.

Q2-2. 일정 주기마다 업데이트한다고 할 때 '일정 주기'는 누가 정하는가?

A. 코딩하는 개발자가 정함.(구현한 코드에서는 1000회당 한번 업데이트함)
또한, 몇회마다 한번 업데이트하는 방식의 Hard Update방식이 있고
매번 복사하지 않고 조금씩 따라가게하는 Soft Update 방식도 있음

Q2-3. 일반적으로 생각해보면 학습 목표가 변하는 것을 방지하기 위해 업데이트하는데 Soft Update가 훨씬 좋은 것 아닌가?

A. 꼭 그렇지만은 않음
Soft Update는 Target Network를 더 부드럽게 만들어줄 수 있지만
그만큼 Target Network가 Q-Network의 변화를 너무 늦게 따라갈 수 있는 단점이 존재함. 따라서 Target Network가 최신 학습 결과를 반영하는 데 시간이 오래 걸리게됨.

Q.2-4 CartPole 문제에서는 어떤 Update 방식이 더 좋은가?

A. DQN의 기본적인 학습 구조를 적용하기에 Hard Update가 더 적절하고 안정성을 확인하기 쉬움. 또한, Action이 비교적 단순한 문제에서는 Hard Update가 더 좋은 편임

Q3. DQN이 가중치를 업데이트하는 방식은 정확히 어떻게 되는가?

A. Target Q값이랑 Q-Network를 통해 출력된 예측 Q값의 오차를 계산하고 그 오차를 손실함수에 넣음. 손실함수의 결과를 역전파로 업데이트시킴.

Q3-1. workflow에 정리한 내용으로는 Q-Network가 두번 쓰이는데 그러면 예측 Q값은 2번 계산되고 Target Q 값은 한번만 계산되는건가?

A. 실제로 두번 계산된다기보단 첫번째 Q-Network에서 계산된 Q값을 저장해놨다가 Target Q랑 계산할 때 사용하는 것임

즉, 한번의 학습 과정 당 Q Network, Target Network 각각 한번씩 계산됨.

Q4. CartPole 문제를 DQN이랑 푸는 것과 PID로 푸는건 어떻게 다를까

A. 제어 입력을 만드는 방식에서 큰 차이가 있음
PID는 목표값과 현재값의 오차를 이용해 제어 입력 계산
DQN은 시행착오를 통해 보상을 최대화하는 행동을 학습함.

Q4-1. DQN도 보상 최대화 학습이라고하는데 결국 그 보상을 업데이트하는게 오차 이용하는거 아닌가?

A. 둘다 오차를 사용하는게 맞지만 오차를 계산하는 기준이 다름
PID에서는 목표 각도가 10이고 현재 각도가 0이라면 L = 10 - 0 으로 계산함
DQN에서는 목표 각도와 현재 각도를 직접 비교하지않음
Q-Network가 예측한 Q값과 벨만방정식으로 계산한 Q값의 차이를 계산

즉, PID는 오차를 계산한 뒤 그 결과로 바로 제어를 하지만
DQN은 오차를 계산한 뒤 그 결과로 신경망의 가중치를 업데이트함

1.3 DQN 개념 요약

1) TD target 수치 계산

   앞서 말한 바와 같이 DQN은 현재 상태에서 선택한 행동의 Q값이 다음 TD target에 가까워지도록 학습함.

2) Replay Buffer 역할

   Replay Buffer는 환경에서 얻은 상태, 행동, 보상 등과 같은 값들을 경험으로 저장함
   학습 시 최근 경험만 순서대로 사용하지않고 저장된 경험 중 일부를 무작위로 뽑아 학습하도록함.

   이로써 과거 경험을 여러 번 재사용해 학습 효율을 높이고 연속된 경험끼리의 상관성을 줄여 학습을 안정화하는 데 도움을 줌

3) Target Network 역할

   TD target 계산에 사용할 Q 값을 제공하는 신경망

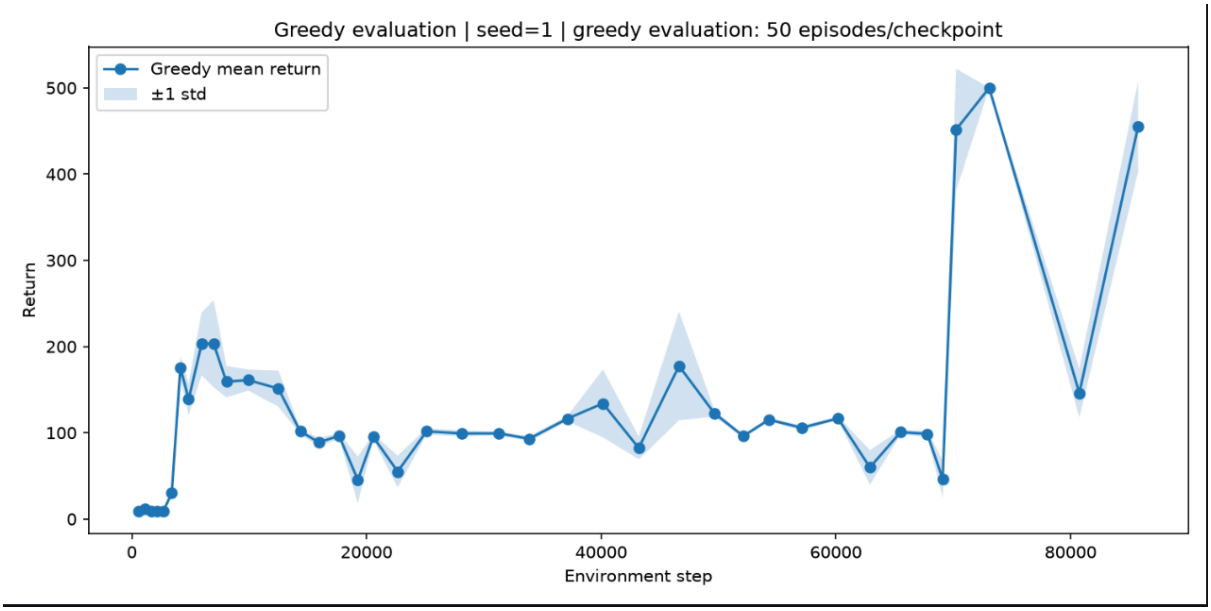
   목표값이 매 순간 함께 변하는 것을 줄여 학습을 안정화시키위해 일정 주기마다 Q Network의 가중치를 복사함

2. 실험 설정 및 재현

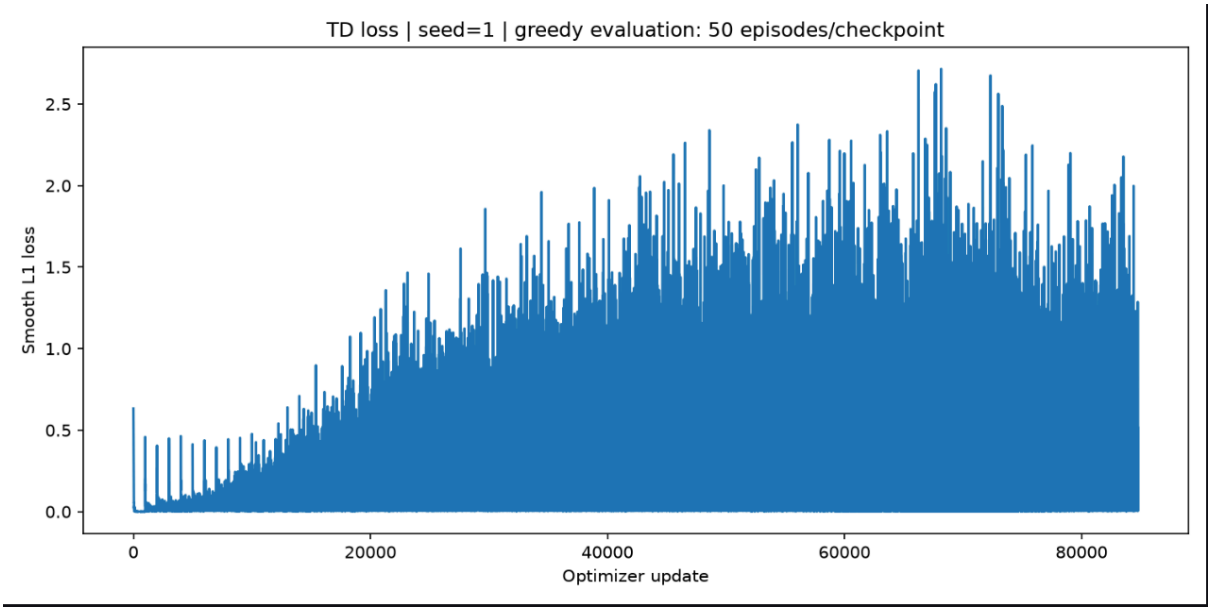
2.1 기본 설정

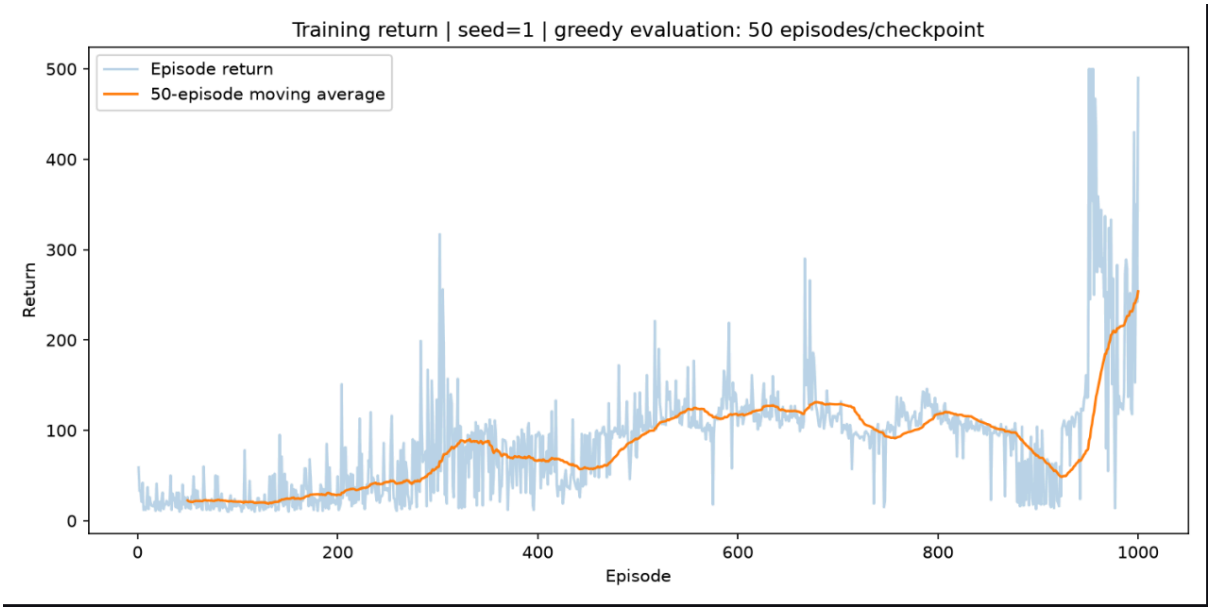
변수	설정
학습 에피소드	1000개
Replay Buffer 용량	50,000
batch size	64
gamma(할인율)	0.98
Learning Rate	0.001
Target Network 갱신	1,000회 당 1회
epsion 감소	환경 step마다 0.0000495씩 감소, 20,000step이후로 0.01 유지
학습 중 평가	25 에피소드마다 greedy 50회
최종 평가	체크포인트 선택과 별도인 seed 50회

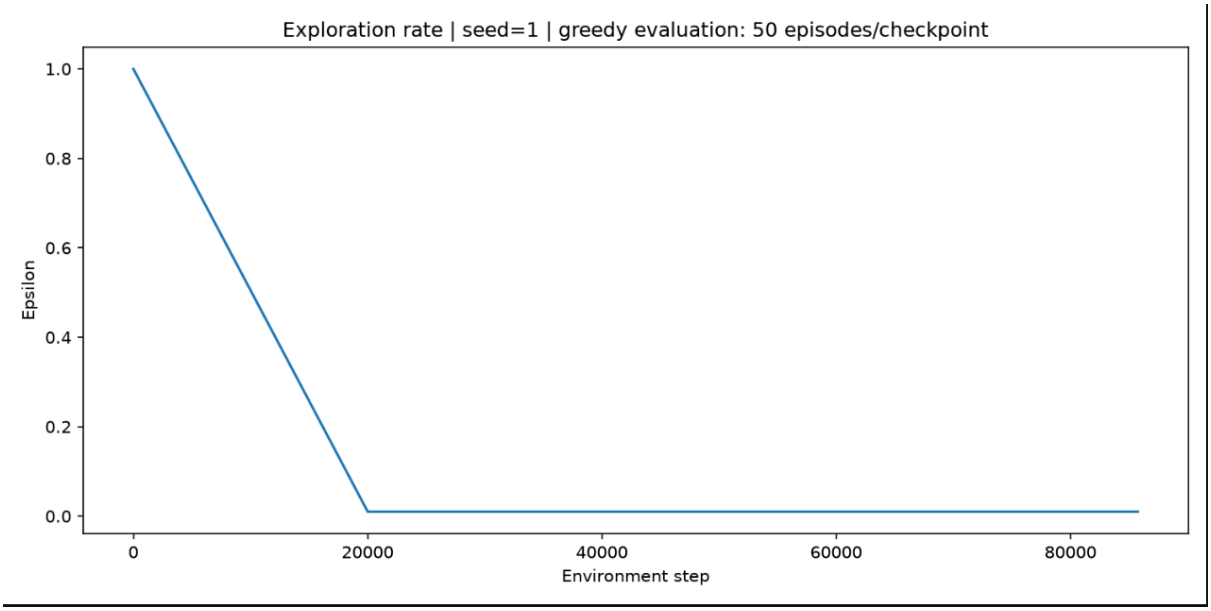
[결과]



greedy 평가: mean=500.00, std=0.00, min=500, max=500







[요약]

greedy 평가 그래프를 봤을 때 마지막에 갑자기 값이 튀고
TD Loss 그래프는 계속 우상향하기에 학습이 잘 되었다고 할 수 없음.

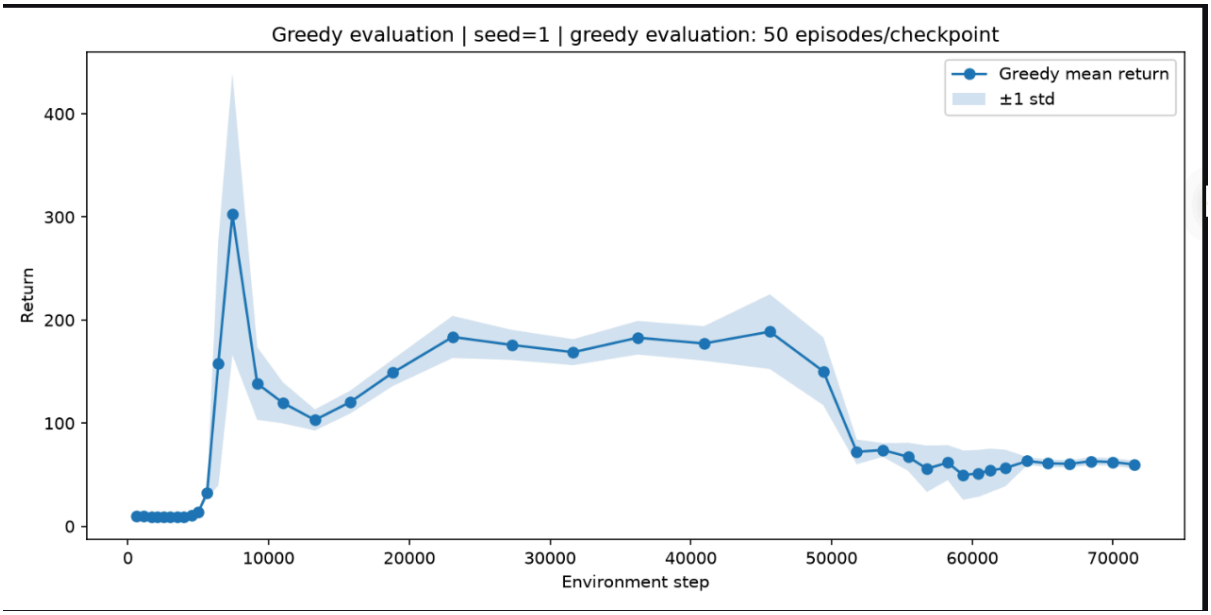
2.2 어떻게 해야 안정적인 학습이 가능할까?

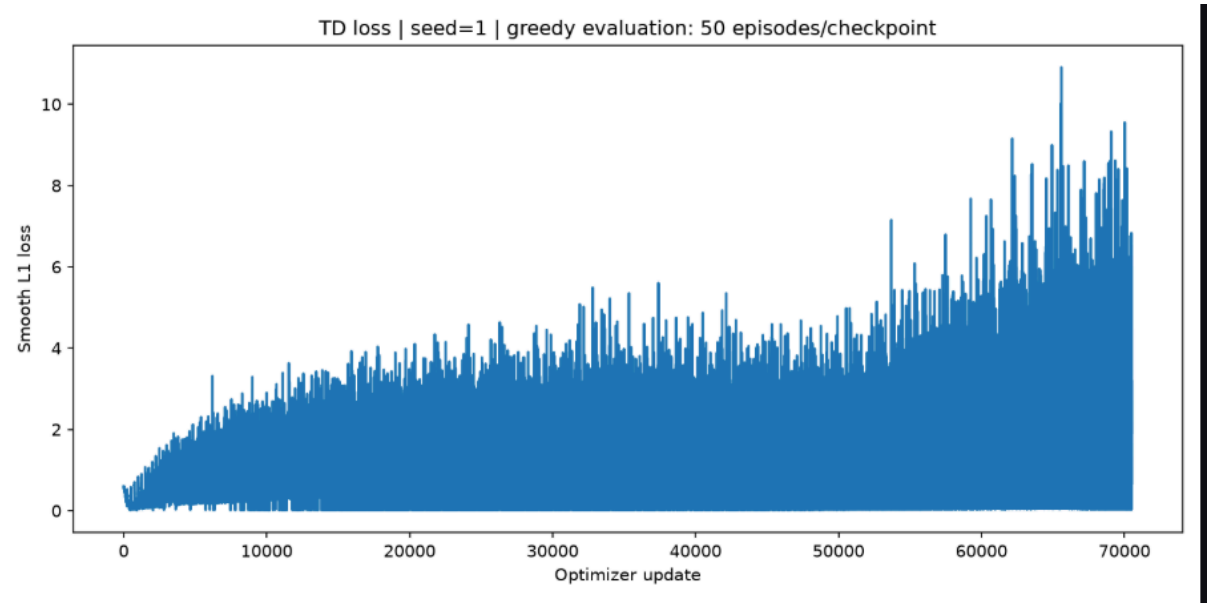
Replay buffer 크기 늘림(혹시나 넘쳐서 덮어씌워질까봐)
학습률 0.0001로 더 낮춰보기(마지막에만 값이 튀어서 최소값에 수렴 안하는것일까봐)

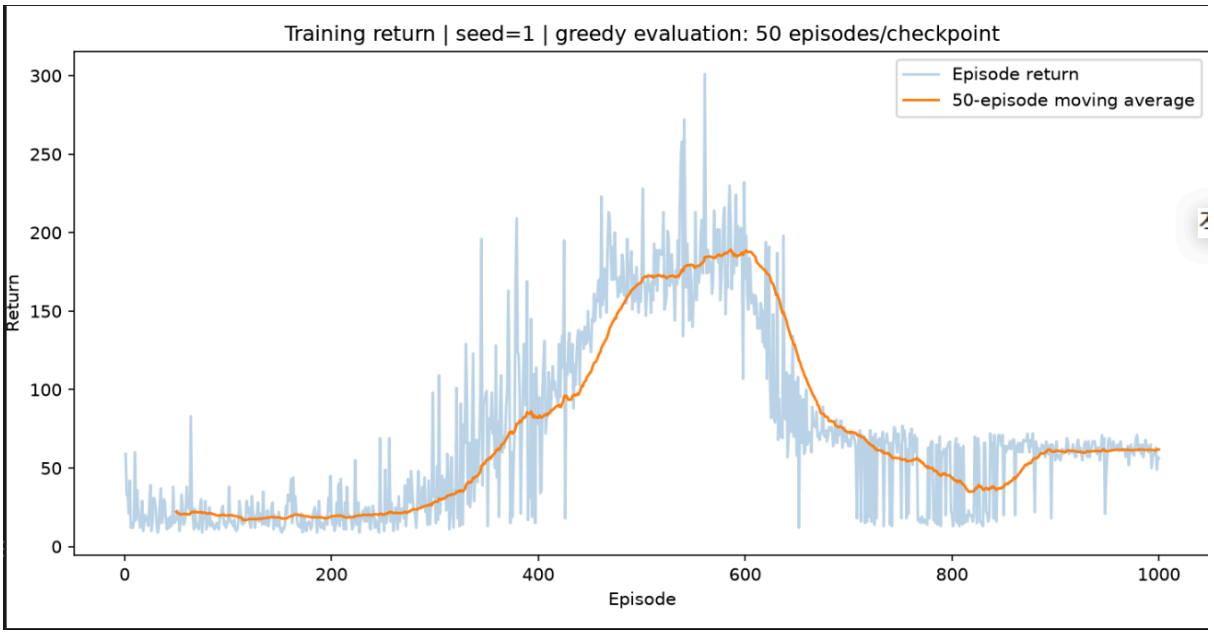
변수	설정
학습 에피소드	1000개
Replay Buffer 용량	**100,000**

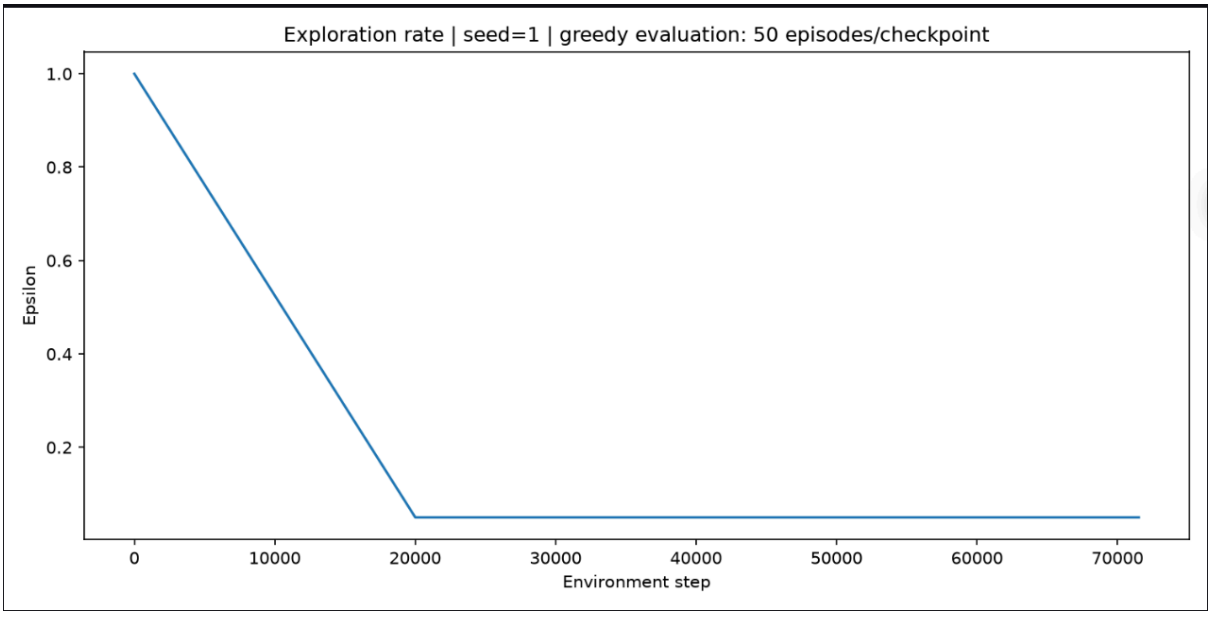
batch size	64
gamma(할인율)	0.98
Learning Rate	**0.0001**
Target Network 갱신	500회 당 1회 && 2000회 당 1회
epsion 감소	환경 step마다 0.0000495씩 감소, 20,000step이후로 0.01 유지
학습 중 평가	25 에피소드마다 greedy 50회
최종 평가	체크포인트 선택과 별도인 seed 50회

[결과]









[요약]

Greedy 평가가 약 50,000 step까지 잘 버티다가 한번 붕괴 이후로 회복하지 못함.
초반에 값이 튄 것은 어쩔 수 없다고 판단
또한, 학습 return 값이 급상승 했다가 급락하는 모양임
policy가 잘 만들어지지 않았다고 판단.

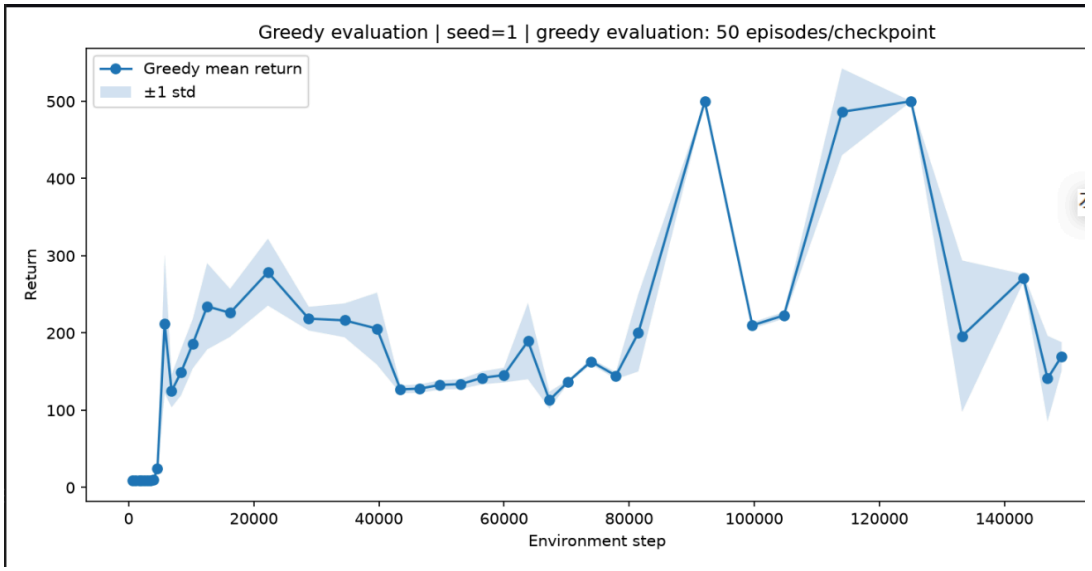
학습률은 불안정의 속도만 늦출 뿐이지 원인을 근본적으로 해결할 수 없다는 것을 알 수 있었음. 또한 버퍼를 늘렸을 때 데이터가 덮어씌워지지않았음에도 학습이 안된건 버퍼의 문제라고할 수 없음.

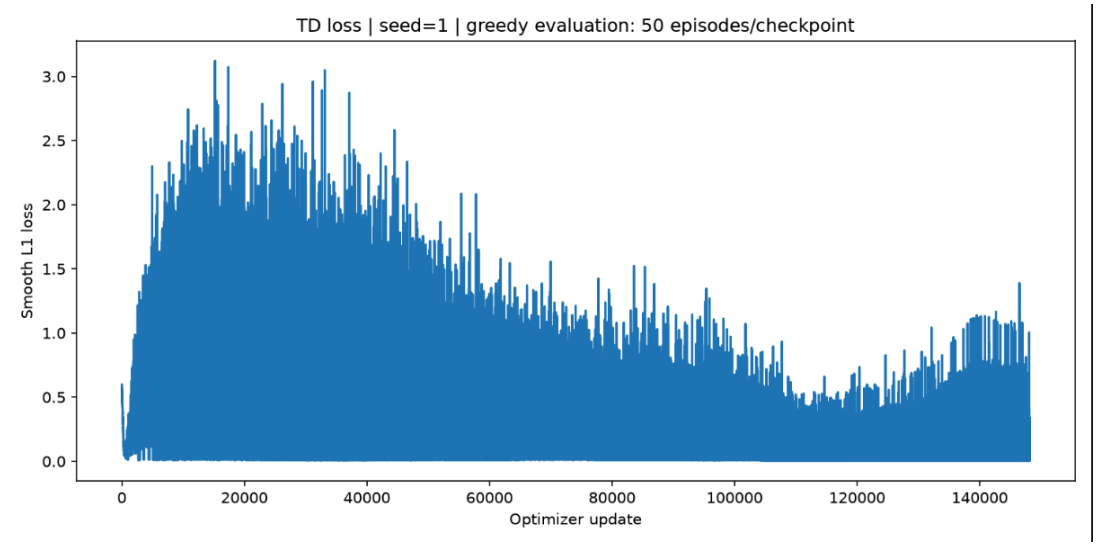
다음 실험에서는 1에서 공부했던 soft update 기능을 추가해보고자함

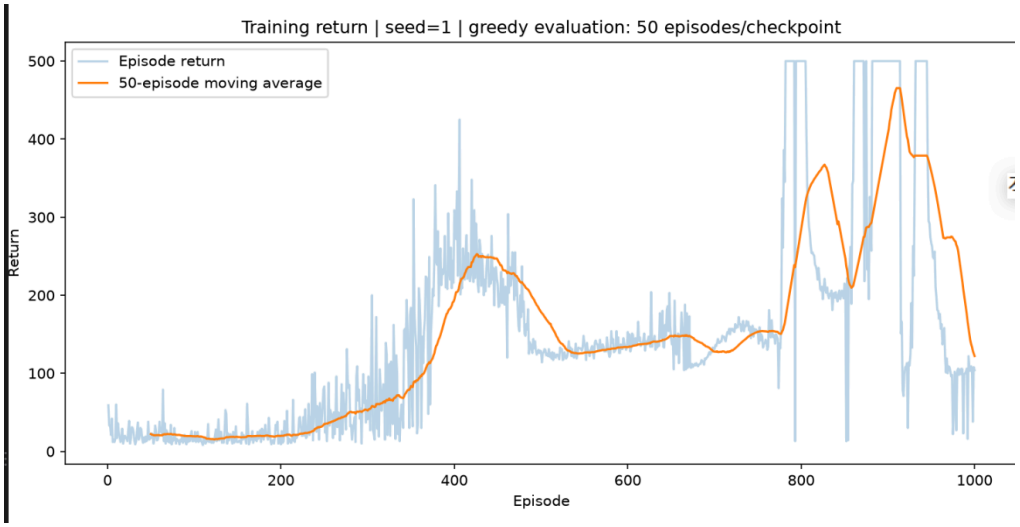
2.3 double DQN과 soft update 추가

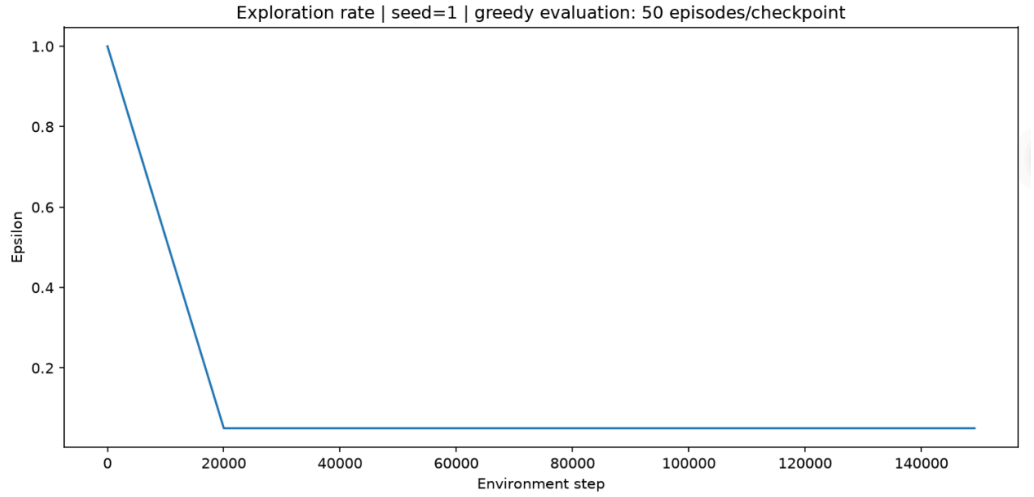
DQN에서 나타난 후반에 성능이 무너지는 문제와 TD loss 상승 문제를 줄이기 위해 Double DQN과 soft update 기능 추가

[결과]
greedy 평가: mean=500.00, std=0.00, min=500, max=500









[요약]

greedy 평가가 약 5,000step까지는 유지되다가 이후 200~280 수준을 유지함. 500점(만점)이 총 3번 발견되었고 그 후 다시 성능이 떨어짐

TD loss는 초반에 최대 3.1까지 올랐다가 계속 감소하는 추이를 보였고 학습 종료 직전에 살짝 증가하였음

결론적으로 TD loss가 안정되었고 만점인 500점에 도달할 수 있게되었음

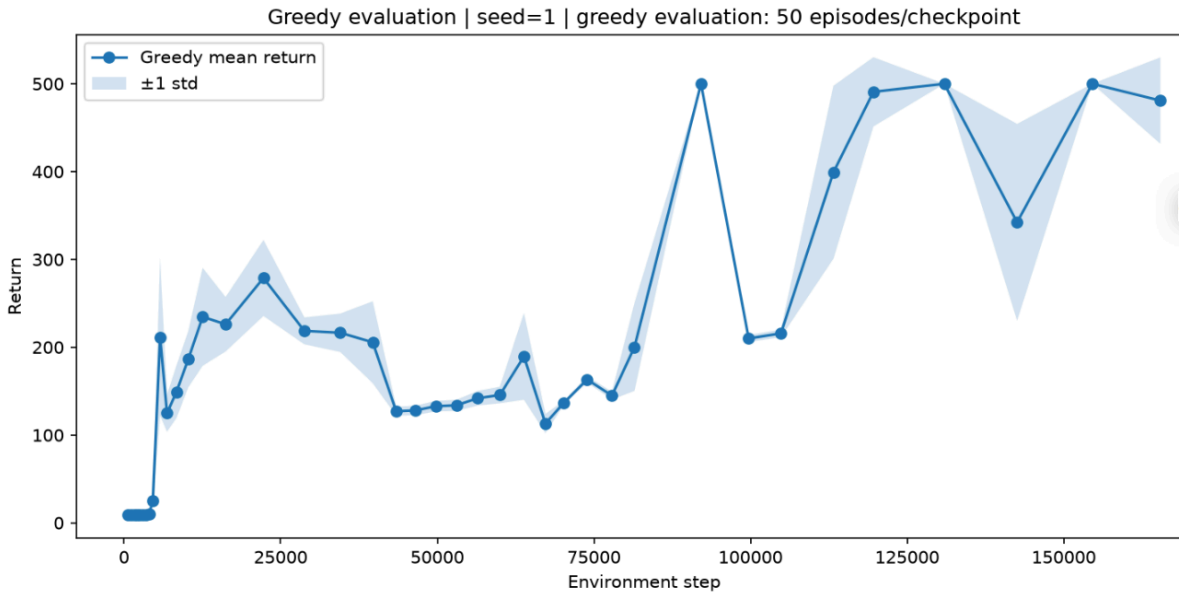
Buffer의 크기를 100,000으로 설정해놓았는데 학습은 140,000step정도 학습되었음. 따라서 다음 실험에서는 Replay Buffer의 크기를 300,000으로 수정.

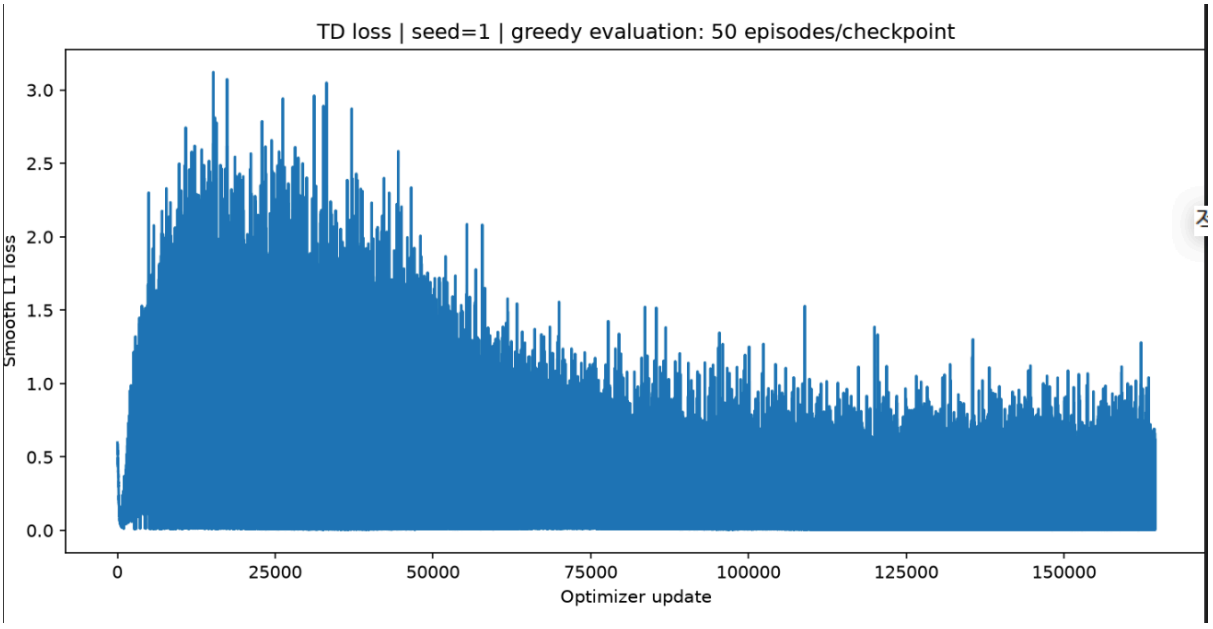
2.4 Replay buffer 크기 수정

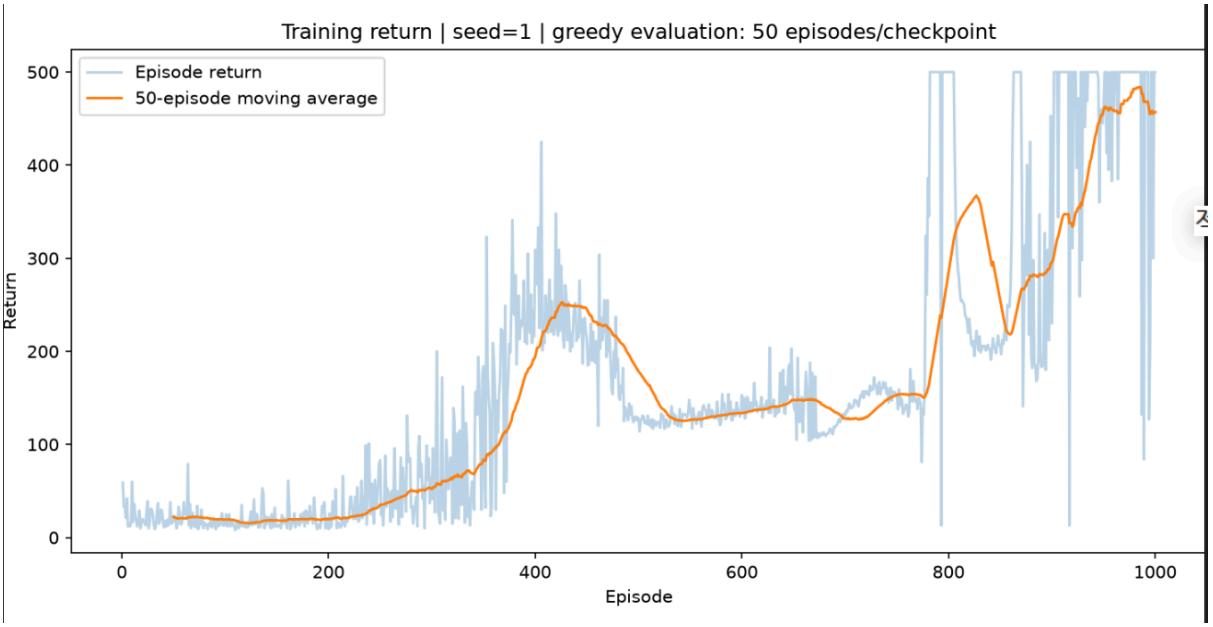
변수	설정
학습 에피소드	1000개

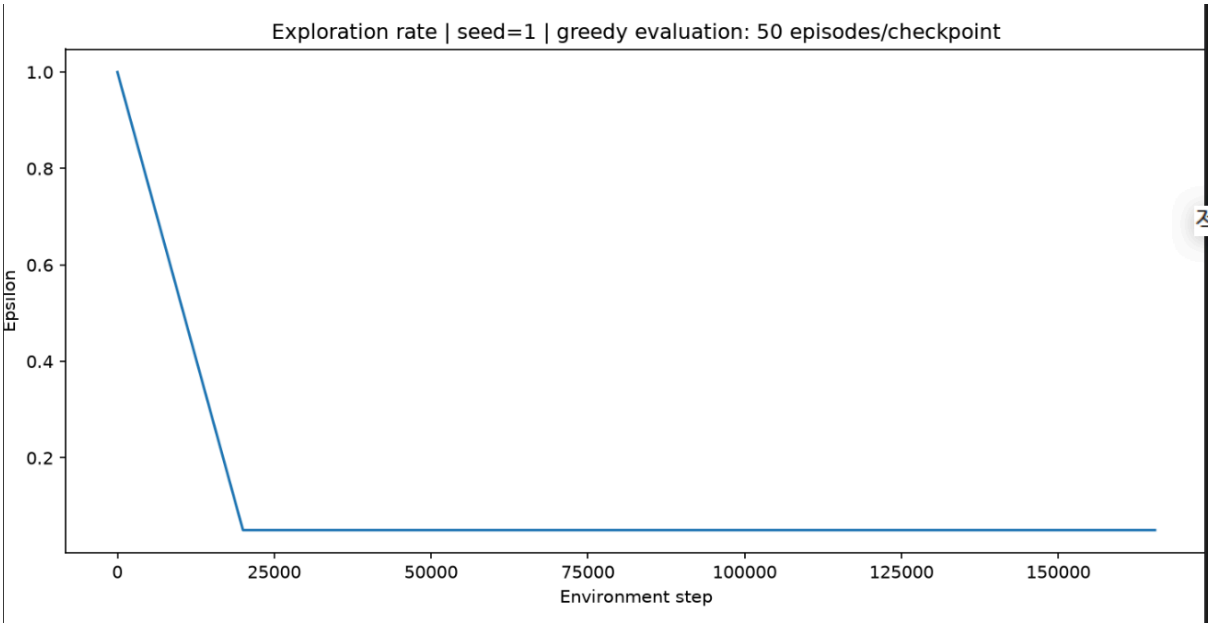
Replay Buffer 용량	**300,000**
batch size	64
gamma(할인율)	0.99
Learning Rate	0.0001
Target Network 갱신	500회 당 1회 && 2000회 당 1회
epsion 감소	환경 step마다 0.0000495씩 감소, 20,000step이후로 0.01 유지
학습 중 평가	25 에피소드마다 greedy 50회
최종 평가	체크포인트 선택과 별도인 seed 50회

[결과]




TD loss | seed=1 | greedy evaluation: 50 episodes/checkpoint
Smooth L1 loss
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
25000
50000
75000
100000
125000
150000
Optimizer update


Training return | seed=1 | greedy evaluation: 50 episodes/checkpoint
Episode return
50-episode moving average
Return
500
400
300
200
100
0
0
200
400
600
800
1000
Episode



[요약]

TD loss가 안정적으로 감소하는 것을 확인할 수 있었고 만점인 500점도 3번에서 5번까지 성공하는 것을 확인할 수 있었으므로 안정적으로 학습되었다고 판단할 수 있음
또한, Training return값도 떨어진다하더라도 모두 잘 회복하였으므로 성공적으로 학습되었다고 판단할 수 있음

2.5 실험 과정 요약 및 느낀점

처음엔 학습 자체가 잘 안되고 TD Loss가 계속 증가하는 그래프가 출력되었음.

buffer크기가 부족하거나 학습률이 너무 커서 수렴하지 않는 문제가 있다고 가정하고
학습률과 버퍼 크기를 수정해보았지만 평균값에 잘 버티는 구간은 존재했지만 500점에 도달하는 경우는 없었고 감소했을 때 다시 회복하는 모습이 보이지 않았음.
다음 실험에서는 1번 궁금증 해결 과정에서 공부했던 soft update 기능을 추가하여 진행해보았고
AI의 추천으로 Double DQN도 추가했음.
max로 타깃을 만들면 Q값이 과대 추정되고 그 값이 다시 타깃으로 쓰이면서 오차가 쌓이는데
이 문제를 해결하기 위해 Q-Network가 고르는 가치는 Target Network가 평가하도록 double DQN을 추가하였습니다.

soft update를 추가하니 TD loss가 안정되고 붕괴된 후에도 회복하며 500에 도달하는 경우가 발생하였습니다. 이때 replay buffer의 크기를 100,000으로 설정했는데 학습은 140,00개가 되어서 여유있게 replay buffer의 크기를 300,000으로 설정했더니 안정적으로 학습이 잘 되었습니다.

이 실험을 해보며 느낀 점은 1번 과정을 공부하며 AI에게 Hard Update가 좋은지 Soft Update가 좋은지 물어봤을 때 Hard Update가 더 좋을거같다는 의견을 주어서 그렇게 코딩하였는데
AI의 말을 무조건 수긍하지않고 다른 시도를 해보는 것도 사고를 확장해 나아갈 수 있는 길이라는 것을 알 수 있었습니다.

3. AI 사용

학습이 잘 되지않을 때 제가 생각한 가정이 맞는지 AI에게 조언을 받았습니다.
예를 들어 “~~값을 수정하면 ~~한 결과가 나올거 같아서 바꿔보려고하는데 맞는 가정이야?”
라고 조언을 구했고 soft update 코드 수정할 때 AI의 도움을 받아 코드 작성하였습니다.